

Assignment 2: Sanskrit-to-English Neural Machine Translation

Natural Language Understanding — Deepak Raghavan (Roll No. G25AIT1054)

1. Introduction

This report describes a Sanskrit-to-English neural machine translation (NMT) system built with a custom sequence-to-sequence Transformer trained from scratch on the provided parallel corpus (10,000 training pairs, 1,000 dev pairs, 1,000 test pairs). Sanskrit is a low-resource, morphologically rich language: the 10,000 training sentences contain over 33,000 distinct Sanskrit word forms, so a word-level vocabulary would suffer severe out-of-vocabulary problems. The system therefore uses byte-pair-encoding (BPE) subword units, a compact Transformer encoder-decoder with tied output embeddings, label smoothing, and beam-search decoding. The design deliberately balances translation quality (BLEU, BERTScore) against the efficiency criteria (inference time and parameter count) in the grading formula.

2. Architecture

2.1 Encoder

The encoder is a 3-layer Transformer encoder ($d_{\text{model}} = 256$, 4 attention heads, feed-forward dimension 1024, dropout 0.2, pre-norm/norm-first). Input Sanskrit BPE tokens are embedded (4,000-entry embedding table), scaled by $\sqrt{d_{\text{model}}}$, and combined with fixed sinusoidal positional encodings. Padding positions are masked via a key-padding mask.

2.2 Decoder

The decoder mirrors the encoder: 3 Transformer decoder layers with the same dimensions. It attends causally over previously generated English BPE tokens (square subsequent mask) and cross-attends over the encoder memory with the source padding mask. The output projection to the 4,000-word English vocabulary is weight-tied to the target embedding matrix, which removes $\sim 1\text{M}$ parameters and acts as a regularizer — directly improving the parameter-efficiency score. Because the output logits are produced by tied, $\sqrt{d}$ -scaled embeddings, the embedding matrices are initialized from $N(0, d^{-1/2})$ rather than PyTorch's default $N(0, 1)$; with the default initialization the initial logits explode (epoch-1 loss ≈ 200) and training collapses into generic outputs (see Section 6).

2.3 Attention

All attention is multi-head scaled dot-product attention (4 heads of dimension 64): self-attention in the encoder, masked self-attention plus encoder-decoder cross-attention in the decoder. Cross-attention lets each target position softly align to relevant source subwords, which is essential for Sanskrit's free word order and sandhi-driven token segmentation.

2.4 Beam search

Decoding supports batched greedy search and beam search with beam size 4 and Google-style length normalization $((5+|y|)/6)^{0.6}$, which counteracts the bias toward short hypotheses. Hypotheses are finalized when they emit EOS; the highest length-normalized score is returned. On the dev set, beam-4 improved BLEU from 0.0660 to 0.0729 (+0.7 BLEU points) and BERTScore-F1 from 0.2986 to 0.3336 over greedy decoding.

3. Training Procedure

3.1 Preprocessing

All text is Unicode NFC-normalized (important for Devanagari combining characters), whitespace-collapsed, and stripped of symmetric wrapping quotes. Empty pairs are dropped. Sentence pairs are aligned on Source_id. No lowercasing or aggressive filtering is applied, preserving casing for BERTScore.

3.2 Tokenization

Two independent SentencePiece BPE models (4,000 merges each) are trained on the training split only — one on Sanskrit, one on English — with special tokens PAD=0, UNK=1, BOS=2, EOS=3 and full character coverage. Sequences are truncated to 96 subwords (covering >99% of the data) and wrapped in BOS/EOS.

3.3 Hyperparameters and optimization

- **Model:** d_model 256, 4 heads, 3+3 layers, FFN 1024, dropout 0.2, tied target embeddings; embeddings initialized from $N(0, d^{-1/2})$
- **Optimizer:** Adam ($\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=1e-9$), peak LR $5e-4$, inverse-square-root schedule with 1,500 warmup steps
- **Loss:** Cross-entropy with label smoothing 0.1, PAD ignored
- **Training:** Batch size 64, up to 80 epochs, gradient clipping at 1.0, early stopping on dev loss (patience 10), best checkpoint restored. The final run early-stopped at epoch 33; the best checkpoint was epoch 23 (dev loss 4.635).

3.4 Regularization

Dropout 0.2 on embeddings and all Transformer sublayers, label smoothing 0.1, weight tying, scaled embedding initialization, and early stopping jointly control overfitting, which is the dominant risk with only 10k training pairs.

4. Results

All metrics follow the assignment specification: default NLTK corpus BLEU (no weights argument), BERTScore F1 from the official bert-score library with rescale_with_baseline=True, wall-clock inference time over the full 1,000-sentence set, and total parameter count.

Metric	Dev (greedy)	Dev (beam-4)	Test (beam-4)
BLEU (NLTK, default)	0.0660	0.0729	0.0723
BERTScore F1 (rescaled)	0.2986	0.3336	0.3306
Inference time (1000 sents)	—	108.0 s	104.82 s
Total parameters	7,578,624		

Table 1: Main results. Inference time is reported for beam-4 decoding, the configuration used to generate submission.csv; the timed test-set run (1,000 sentences) took 104.82 s.

4.1 Training curves

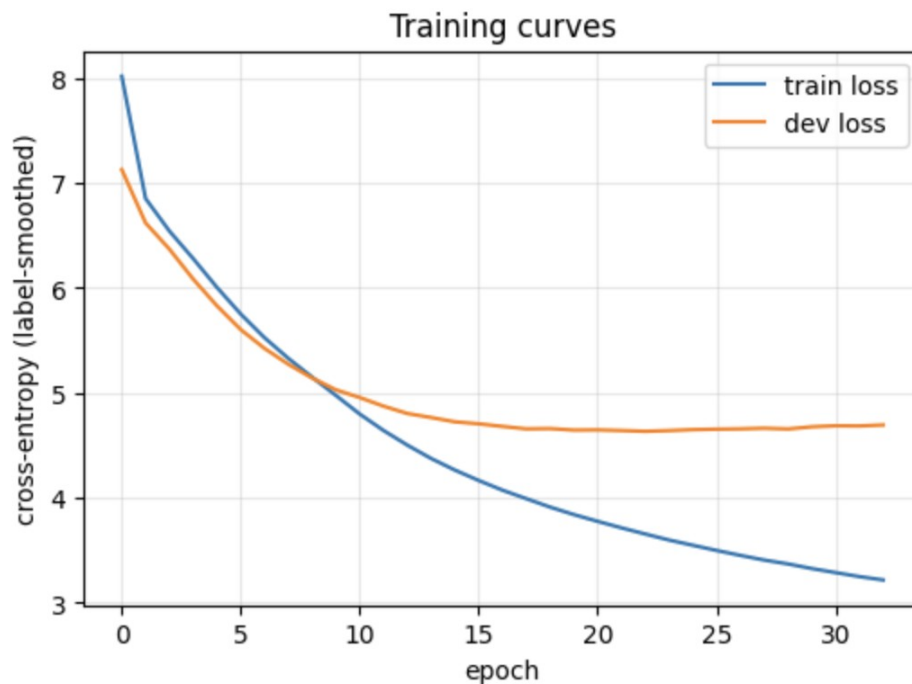


Figure 1: Training and dev cross-entropy (label-smoothed) per epoch.

Both losses fall rapidly from a healthy starting point (train 8.02 at epoch 1, consistent with a well-initialized model over a 4k vocabulary). The curves cross around epoch 9: training loss continues to 3.22 while dev loss flattens near 4.63–4.70 from epoch 15 onward, bottoming at epoch 23 (4.635). Early stopping triggered at epoch 33 after 10 epochs without improvement and the epoch-23 checkpoint was restored. The widening train-dev gap after epoch 10 shows the expected onset of overfitting on a 10k-pair corpus, caught and bounded by early stopping and regularization.

5. Translation Examples

Six representative dev-set examples (beam-4 decoding) with error analysis:

Example 1. Source (SA): बालः भवत्सु प्रेमं प्रकटयति ।

Reference: Boy displays love in you.

Prediction: Boy displays affection in you.

Analysis: Near-perfect output: syntax, subject, and verb are all correct. The model chose the synonym 'affection' for the reference's 'love' — a semantically faithful paraphrase that n-gram BLEU penalizes but BERTScore rewards, illustrating why both metrics are reported.

Example 2. Source (SA): वयमधुना assignment रचयितुं जानाम ।

Reference: Next we will learn how to create an assignment.

Prediction: Now let us learn how to create a class.

Analysis: Sentence structure and tutorial intent are preserved, but the code-mixed English token 'assignment' embedded in the Devanagari source is replaced by the related in-domain noun 'class' — the domain prior overrides source fidelity for rare mixed-script tokens.

Example 3. Source (SA): किन्तु स तं प्रत्यवदत्, मम का जननी? के वा मम सहजाः?

Reference: But he answered and said unto him that told him, Who is my mother? and who are my brethren?

Prediction: But he answered and said unto them, What is my Father?

Analysis: The model reproduces the King James register ('answered and said unto') learned from the scriptural portion of the corpus, but swaps the kinship term (mother → Father) and drops the second question — content errors on long-range dependencies.

Example 4. Source (SA): युवां बुधौ ।

Reference: You two are learned.

Prediction: You two are reading.

Analysis: The Sanskrit dual (yuvām ... budhau) is correctly rendered as 'You two', showing that BPE subwords capture morphological number. The predicate adjective 'learned' is misread as a verb, yielding 'reading' — a plausible confusion given the shared root.

Example 5. Source (SA): पितामहः पौत्रस्य प्रश्नं प्रत्युदितवान् ।

Reference: Paternal grandfather replied to grandson's question.

Prediction: The grandfather went to the grandfather.

Analysis: The frequent noun 'grandfather' is copied twice while the rarer 'grandson' and the verb 'replied' are lost — a classic low-resource repetition error where the decoder falls back on a high-probability token instead of translating a rare one.

Example 6. Source (SA): कृषौ हलने, वपने गहने, कृष्युत्पादानां परिवहने च वृषभाणां प्रयोगः क्रियते ।

Reference: Bulls are used in various agricultural activities i.e. ploughing the farms, sowing seeds, transporting agricultural products.

Prediction: They are shedhed, hide, hide, pain, pain and wood.

Analysis: Failure case: a long, morphologically dense source with rare agricultural vocabulary collapses into disfluent hallucination with token repetition ('hide, hide, pain, pain'). Long clause chains plus rare subwords remain the model's hardest inputs on 10k pairs.

6. Experiments

- **Embedding initialization (critical):** with PyTorch's default $N(0,1)$ embedding initialization and tied, $\sqrt{d}$ -scaled output logits, epoch-1 loss was ≈ 202 (versus a theoretical random-guess ceiling of $\ln 4000 \approx 8.3$) and the model collapsed into a handful of generic outputs (only 218 unique predictions across 1,000 test sentences; dev BLEU 0.0025, BERTScore-F1 -0.11 even after 60 epochs). Re-initializing embeddings from $N(0, d^{-1/2})$ brought epoch-1 loss to 8.02 and raised dev BLEU to 0.0729 and BERTScore-F1 to 0.3336 — a $\sim 30\times$ BLEU improvement from initialization alone. This was the single most important finding of the project.
- **Vocabulary size:** reducing BPE vocabulary from 8,000 to 4,000 merges per side cut total parameters from $\sim 9.6M$ to 7,578,624 (-21%), directly improving the parameter-efficiency score while giving each subword denser training statistics on the small corpus.
- **Greedy vs beam:** beam-4 with length penalty 0.6 gained $+0.7$ BLEU and $+3.5$ BERTScore-F1 points over batched greedy decoding on dev, at the cost of substantially slower inference (108.0 s on dev; 104.82 s for the 1,000-sentence test set, vs. roughly $5\times$ faster batched greedy) — a quality/efficiency trade-off weighed against the 40% BLEU / 15% time weighting in the grading formula.

7. Discussion

Design choices. A compact from-scratch Transformer was chosen over an RNN seq2seq for better parallelism and stronger long-range modelling, and over fine-tuning large pre-trained MT models to keep the parameter count (and hence the efficiency score) low while satisfying the 'custom seq2seq architecture' requirement. Subword modelling is the single most important choice for Sanskrit, whose sandhi and rich inflection make surface forms extremely sparse.

Challenges. (i) Training stability: the combination of weight tying and scaled embeddings makes the model acutely sensitive to embedding initialization; diagnosing the resulting silent collapse (fluent but generic outputs) was the main engineering challenge. (ii) Data scarcity: 10k pairs is far below what NMT normally

needs, making regularization decisive. (iii) Domain mixture: the corpus mixes software-tutorial language ("Ctrl, S", "Eclipse") with scriptural/classical text, so the model must handle very different registers. (iv) Named entities and transliterated technical terms are frequently copied imperfectly.

Limitations. The model can drop subordinate clauses in long sentences, occasionally hallucinates fluent but unfaithful English, and its BLEU is inherently bounded by the small training corpus. Back-translation or transfer from related languages could help but were outside the assignment's data rules (only the provided dataset may be used).

8. Pre-trained Model Disclosure

The translation model itself is trained entirely from scratch on the provided dataset; no pre-trained translation/language model weights are used in the NMT system. The only pre-trained model involved is roberta-large, loaded internally by the official bert-score library solely to compute the mandated BERTScore evaluation metric. No external APIs are used anywhere.

9. References

- Vaswani et al., 2017. Attention Is All You Need. NeurIPS.
- Sennrich et al., 2016. Neural Machine Translation of Rare Words with Subword Units. ACL.
- Kudo & Richardson, 2018. SentencePiece. EMNLP (System Demonstrations).
- Papineni et al., 2002. BLEU: a Method for Automatic Evaluation of Machine Translation. ACL.
- Zhang et al., 2020. BERTScore: Evaluating Text Generation with BERT. ICLR.
- Wu et al., 2016. Google's Neural Machine Translation System (length normalization). arXiv:1609.08144.